LPF: mask [0:4] = 1 , Keep DC to 12 KHz. 10
mask [5:15] = 0 : [zero, for higher freq].

*HPF: mask [0], Kill DC.
mask [1:2] = 0,
mask [3:15] = 1, Keep High freq.

*BPF: Keep mid frequ.
mask [2:6] = 1 (~6-18 12 Hz)
mask [0:1, 7:15] = 0.

→ what's the math behind it? (Bin)

$f_K = K * (F_s / N)$ → No. of Samples

f_K: Freq. we want
K: mask (0, 1)
F_s: sampling freq (given i/p freq)
N: No. of Samples

* if F_s = 48 KHz, N = 16;

Bin 0 = 0 Hz, Bin 1 = 3 KHz., bin 2 = 6 Kz,
... Bin 8 = 24 KHz (Nyquist no.)
Bin 9-15 - mirrored negative fr.

Now we know which one we want & we do.

realme Shot on realme GT NEO 3T
2026 04 11 21:55

ifft_core.v

Converts fft output which is in frequency domain into time domain.

$$x[n] = \frac{1}{N} \sum_{K=0}^{N-1} X[k] e^{+j2\pi Kn/N}$$

IFFT: has +ve $e^{j2\pi Kn/N}$

* How CKt understand ifft (freq → time).
  - Takes numbers from memory, (represent freq bin)
  - Multiplies with twiddle factor (cos & sin)
  - Runs butterfly.
  - o/p numbers.
  - Twiddle is rotated counter clockwise (~~CW~~)

and

Latency : ~3 cycles (mirrors fft_core.v)

total pipeline < 10 cycles

N=16 - real time (48KHz audio : 333 us /frame).

realme
Shot on realme GT NEO 3T
2026 04 11 21:55

* twiddle for index K in N-point FFT is

$$W_N^K = \cos(\theta) - j\sin\theta$$

$$\theta = 2\pi \frac{K}{N}$$

* for N=16, $\theta = 22.5^\circ$ $(360/16)$ when K=1.

!!) $\cos(22.5) = 0.9239$

$\sin(22.5) = 0.3829$

- $0.9239 - j0.3829$

* in our code : $\sin(\pi/4) = 45^\circ \Rightarrow 0.7071$

* coding:

i/p: $(\text{clog2}(N)-1:0) = \log_2(16)=4) : 4$ bits

a) bit_reverse.v:

when we perform DIT, o/p are in scrambled form. Specifically they are in bit-reversed order compared to the natural frequency

* i/p: input wire signed [N* DATA_WIDTH-1:0] img_in;
input wire signed [N* DATA_WIDTH-1:0] real_in.
, holds all real part concatenated.

* o/p: output reg signed [N* DATA_WIDTH-1:0] real_out;

* logic: localparam INDEX_WIDTH = $clog2(N);
$\log_2(16) = 4$, INDEX_WIDTH = 4
since 4 bits required to count 0 to 15

realme Shot on realme GT NEO 3T
2026 04 11 21:55

8) ALU stage:
Simple, does add, sub, comparision, etc.

• The FFT basically converts this input sample that are in time domain into frequency domain.
↳ what frequencies are present
↳ how strong they are

$$X[k] = \sum_{n=0}^{N-1} x(n)\, e^{-j 2\pi k n / N}$$

$e^{-j 2\pi k n / N}$ → twiddle factor.

• Here in our fft_core.v, time domain samples ($x[n]$) are multiplies with the twiddle factors (butterfly addition), produces frequency domain representation $X[k]$.

* Radix 4:
• Sorting 4 cars at a time, but it only works if you have no. of cars that's power of 4 (4, 16, 64).

* DIT (Decimation in time):
It's the order the robot sorts.
• For N-point FFT, the no. of stages is $\log_4(N)$
So like $\log_4(16) = 2$ stages, if we use $\log_2(16) = 4$ ↳ radix 2

* DIT (Decimation in time):
• DIT starts with time-domain and splits them into group based on their time order. (odd & even).
• Each stage goes under butterfly operation (+, −, * with twiddle factor).

realme Shot on realme GT NEO 3T 2026 04 11 21:55

FIR:
· multiplies each TAPS with past TAPS coefficients. then add.
· when 'start_fir' is 1, module walk through all tap multiplies.
'input_buffer[t]' coeff[t], = acc' & write the result in 'fir_output' & raises 'fir_done'.

$$y = \sum_{K=0}^{TAPS-1} h[K] \cdot x[K]$$

↳ we go with 16 tap (smooth & simple to implement)

⑦ fft_core.v:
"chiplet" uses FFT to figure what frequencies are present in the signal.
· DATA WIDTH = 16 : each sample is 16 bit wide.
N = 16; 16 point FFT.

→ i/p:
· real_in: Array of input samples [Real part]
· img_in: Array of input samples [imaginary part]
· real_out[]:

realme
Shot on realme GT NEO 3T
2026 04 11 21:55

⑥ fir_filter.v: perform convolution between input samples & set of fixed coefficients

- Low pass, high pass, band-pass filtering
- Smoothing signal
- Removing noise
- Implemented as "Multiply accumulate (MAC) pipe

→ what's FIR?

Digital filter that take recent samples, multiplies each sample with "coefficient" & adds them all together to produce new output.

$$y = x_0h_0 + x_1h_1 + x_2h_2 + \dots + x_nh_n$$

(x: samples, h: coefficients)

- Low pass filter: Removes high frequency signal → noise is a high freq.
- High pass filter: Removes low freq. signal → DC offset

→ coefficients in FIR:

- These are some fixed vals, designed based on the desired frequency response.

↳ This designs is done by Windowing method:

$$h(n) = \frac{\sin(2\pi f_c(n - m/2))}{\pi(n - m/2)} \; w(n)$$ → sinc. func

↳ LPF impulse respon.

→ window function: makes filter finite & practical.

↳ config_mode = 5'b00000 → LPF
5'b00001 → HPF
5'b00010 → BPF

→ input wire [DATA_WIDTH-1:0] input_buffer [0:TAPS-1]
16 samples each of 12 bits — no of samples

realme Shot on realme GT NEO 3T
2026 04 11 21:54

dma_controller.v : FSM:

Once the FIR/FFT are over, the DMA controller takes the o/p block & pushes it to memory or o/p buffer.

1) Waits for signal from controller (start_dma_out)
2) Read data from o/p buffer.
3) Pushes that to mem, bus, testbench.
4) Signals when process is completed (dma_done).

o/p →

- dma_data_out: Actual data to be written into mem
- dma_valid: valid signal for output.
- dma_done: goes high when data is dispatched

FSM:-

dma_ptr: holds the samples & updates on every clk pulse.
dma_active: Flag to indicate DMA is busy.
dma_valid: Goes high when "dma_data_out = 1"
dma_done: Signals completion of transfer.

- if (start_dma_out && !dma_active):
  when DMA is not active, mainly starts.
- if (dma_active):
  - Send the final result to o/p.
  - Makes valid = 1.
  - Increments to next pointer.
    - if (dma_ptr == BLOCK_SIZE-1):
      we send all the data, so done = 1;

realme Shot on realme GT NEO 3T 2026 04 11 21:54

③ config_register.v: CONTROL STATUS REGISTER.

- Its a control knob for tuning pipeline,

It store the configuration (FFT/FIR) and other mods like IFFT bypass etc.

→ i/p:

write_enable: load signal: update the value.

config_in : 5bit : control word.

→ o/p:

config_mode : 5bit:

0 : FIR/FFT select
1 : IFFT
2 : Bypass (skip process)
3 : Debug (
4 : DMA (allow DMA dispatch)

This is crucial because only with the help of "configuration register" the DSP knows what operation is been going to performed.

④ clock_manager.v:

Generating & managing & the clock signals for different parts of DSP chiplet

- Accepts "master clock" (100MHz)
- Generate derived clock "clk_fin, cbt, dma".
- Turn off clock when clocks modules are idle.

clk_div2 => toggle every cycle. } For slow down purpose
clk_div4 → toggle every 4 cycle. }

Clock gating => Save power & prevent unnecessary toggelling.

realme Shot on realme GT NEO 3T
2026 04 11 21:54

ontroller.v (FSM)

This is the control unit of DSP chiplet.
monitors when input data is ready / buffer stat.
Decides whether to trigger FFT / FIR
- Signals output DMA to dispatch result.
- Keep track of pipeline activity.

→ i/p:
- ready_for_processing: Tells the controller that buffer is full.
- fft_done: Signal that FFT is completed.
- fir_done: Signals that FIR module is completed.
- config_mode: choose FFT / FIR (dynamic routing)
  0 = start_fir
  1 = start_fft

→ o/p:
- start_fir: Goes high for 1 cycle to trigger FIR engine.
- start_fft: Go " " " " " FFT engine.
- start_dma_out: Tells DMA to dispatch processed block.
- processing_active: goes high when pipeline is busy (stall)

→ FSM:
STATE_IDLE: waits still buffer to fill
STATE_START_PROC: Triggers FIR / FFT
STATE_WAIT_DONE: Waits the FIR / FFT to finish
STATE_DISPATCH: Signals DMA to send O/P.

realme
Shot on realme GT NEO 3T
2026 04 11 21:54

1) input_buffer.v

- Collects incoming samples (from ADC / DMA)
- Stores them in a buffer (FIFO)
- Track how many samples have arrived.
- Signals when buffer is full & ready for processing.

module input_buffer (
parameter DATA_WIDTH = 12, // 12-bit samples
parameter BUFFER_SIZE = 256, // no. of samples
)(

- valid_in : I/P = signals buffer that be reads for new samples.
- sample_in : 12 bit := Actual incoming samples.
- ready_for_processing : 1 = when buffer is full to trigga FFT / FIR
- reg [$clog2(BUFFER_SIZE)-0] write_ptr
  ↳ write points // point where the next sample should be stored.

↳ clog2(256) = 2^8 = 256.
↓
sets the pointer connects to 8-bits.

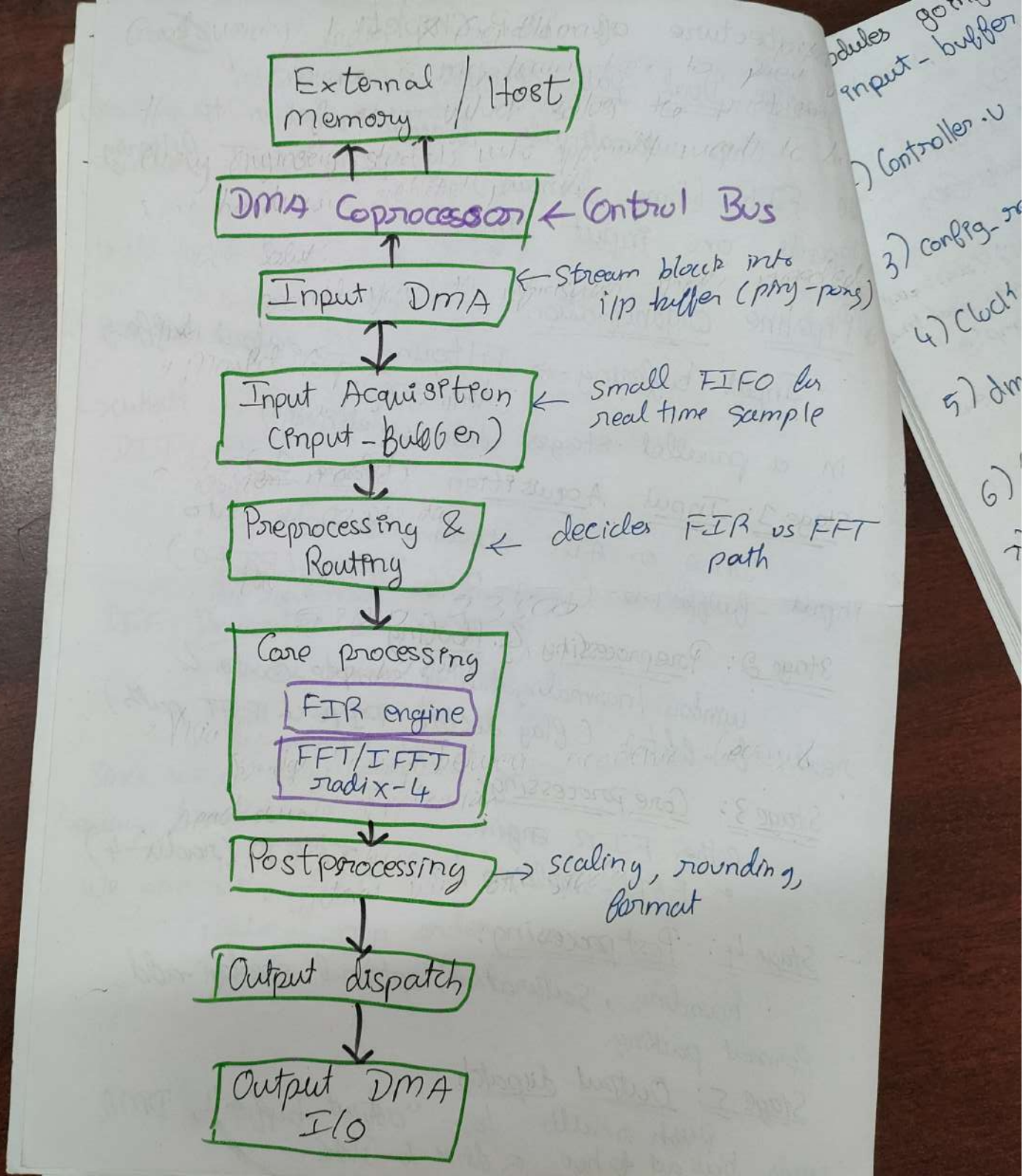
Ex (log2(1024) -> it calculat 2^{10} = 1024
↳ so 8 bits.

(0-prevents from mis arrays of any byte

Modules going to be in our project.

1) input_buffer.v – Stores incoming samples from DMA.
2) Controller.v – FSM that routs data, triggers processing.
3) Config_register.v – Holds FFT size, filter type, mode.
4) Clock_manager.v – optional clock gating or pipelining.
5) dma_controller.v – Handles memory transfer autonomously.
6) fir_filter.v – Time domain filtering using MAC.
7) coeff_rom.v – FIR filter coefficients.
8) delay_line.v – Sample history of FIR.
9) fft_core.v – Radix-4 DIT FFT engine.
10) twiddle_rom.v – Twiddle factor for FFT.
11) bit_reversal.v – Reorder FFT output bins.
12) ifft_core.v – Converts from freq to time domain.
13) freq_filter.v – Frequency domain masking.
14) output_buffer.v – Stores final processed data.
15) interface.v – Sends output to DAC, host/mem.
16) dsp_testbench.v – Simulate full pipeline & benchmark.

realme
Shot on realme GT NEO 3T
2026 04 11 21:54



realme Shot on realme GT NEO 3T
2026 04 11 21:54

Architecture of DSP Chiplet (3)

* Adaptive Dual path design:

Chip automatically decides whether to use FIR (time domain) or FFT/IFFT (filtering) depends on input size.

* Pipeline organization:

Input buffering → Filtering (FFT/FIR) → output buffering in a parallel stages. (data pipelined)

Stage 1: Input Acquisition (Stage 1)

DMA or ADC streams samples into input_buffer.v (ping-pong BRAm / FIFO).

Stage 2: Preprocessing & Routing

window/normalization + sample_counter & reads_en_left. (flag decide FIR or FFT path)

Stage 3: Core processing:

either FIR engine.

or FFT → multiply (4) → IFFT (radix-4)

Stage 4: Post processing:

Rounding, Saturation, optional overlap-add, Format packing.

Stage 5: Output dispatch

Push results to "output_buffer", DMA reads block out to host or driver to DAC.

realme Shot on realme GT NEO 3T 2026 04 11 21:54